

Homework8 题目勘误与修改建议

Problem 2 原文中那句

假设更新 page 都是连续写入一个 block 的，不论更新 page 是来自于哪个 block 。

并不是写错了，它描述的是一种真实存在、已发表的 FTL 算法（FAST）。而课件（2-10-io.pdf 第 30–36 页）所教的“每个原始 block 各对应一个 log block”的方法是另一种真实存在的算法（BAST）。两者都是文献中的标准做法，本身没有矛盾。

Problem 2 真正的毛病在于题面没有把 FAST 需要的参数交代清楚，导致答案不唯一。

背景：log-block 式 hybrid mapping

闪存不能原位覆盖，更新一页必须写到别处，因此 FTL 对绝大多数 block 用块级映射（block table），只对正在被更新的少数 block 额外开 log block 并用页级映射（log table）记录其中每个有效页的位置。访问时先查 log table，命中用最新版本，否则查 block table 读原始数据。当 log block 用完或需要回收时，要把 log block 与对应的原始数据合并（merge），合并分三类：

```
switch merge : log block 恰好按序含某数据块的全部页 -> 直接切换块映射，擦 1 块  
partial merge: 把旧数据块中未修改的页补拷进 log block 凑齐 -> 切换映射，擦 1 块  
full merge   : 有效页分散，需收集到一个新空白块 -> 搬移整块页，擦 2 块
```

BAST 与 FAST 的区别只在于“一个 log block 能为哪些数据块服务”，以及随之而来的 log block 管理与回收策略。

课件方法是 BAST（Block Associative Sector Translation, IEEE TCE 2002）

BAST 出自 Jesung Kim, Jong Min Kim, Sam H. Noh, Sang Lyul Min, Yookun Cho, “A space-efficient flash translation layer for CompactFlash systems,” IEEE Transactions on Consumer Electronics, vol. 48, no. 2, pp. 366–375, May 2002。

它的核心约束是块关联（block-associative）：每个 log block 只服务一个原始数据块，一个 log block 里的更新页只能来自同一个数据块。当写入某逻辑页时，若它所属的数据块已经有 log block，就把新版本追加进该 log block 的下一个空位；否则分配一个空白 log block 与该数据块绑定。系统同时只允许少量 log block（即同时只允许少量数据块处于“被更新”状态）。当没有空闲 log block 又要更新一个新的数据块时，选一个 victim log block 与它绑定的数据块合并，腾出空间。因为 log block 与数据块一一对应，合并时页大多落在正确偏移上，所以容易触发代价低的 switch / partial merge。

课件第 34 页的课堂练习把这套规则写得很清楚：同时最多只有 1 个 log block 存在，每个原始 block 对应一个 log block（一个 log block 中的数据不能来自不同的原始 block）。这正是 BAST。

BAST 的弱点是 log block thrashing：在随机写、且同时涉及很多数据块的负载下，每个 log block 往往只写了一两页就因为要更新别的数据块而被迫合并，log block 利用率低、合并频繁，性能下降。

题面方法是 FAST（Fully Associative Sector Translation, EUC 2006）

FAST 出自 Sang-Won Lee, Dong-Joo Park, Tae-Sun Chung, Dong-Ho Lee, Sangwon Park, Ha-Joo Song, “A log buffer-based flash translation layer using fully-associative sector translation”（EUC 2006，扩展版见 ACM Transactions on Embedded Computing Systems, vol. 6, no. 3, 2007）。

FAST 把 log block 改为全关联（fully-associative）：一个 log block 可以容纳来自任意原始数据块的更新页，更新页按到达顺序连续追加写入当前 log block，不论它来自哪个数据块。这恰好就是 Problem 2 那句“连续写入一个 block，不论来自哪个 block”。这样做让 log block 的每一页都能被利用，利用率高、合并次数少，正好补上 BAST 的短板。

完整的 FAST 把 log block 分成两类：一个 sequential log block（SW，专门承接顺序写，从偏移 0 开始连续写满即可 switch merge，代价低），以及若干个 random log block（RW，全关联，承接随机写）。当需要空白块而没有空闲 log block 时，FAST 在 random log block 中按 round-robin（先进先出）选出 victim 进行回收。

FAST 的代价集中在合并上：一个 random log block 里的有效页可能分属许多不同数据块，合并它时要对其中涉及的每一个数据块各做一次合并，而且某个数据块的最新页可能散落在 victim、别的 log block、以及旧数据块里，必须逐一收集到新块——这通常是 full merge，且会顺带把别的 log block 里属于该数据块的页变为失效。FAST 用“较贵但很少发生的合并”换“高利用率、低合并频率”，在随机负载下整体优于 BAST。

两者对比：

维度	BAST	FAST
log block 关联性	块关联，一个 log block 只服务一个数据块	全关联，一个 log block 可服务任意数据块
log block 利用率	低（易半空就被迫合并）	高（连续追加写满）
合并频率	高	低
单次合并代价	低（多为 switch / partial）	高（多为涉及多个数据块的 full merge）
主要弱点	随机写下 log block thrashing	单次 full merge 贵、最新页分散难追踪

题面真正的问题

问题一：应假设块数无限，图上画 4 个块不代表只有 4 个块

FAST 下 log block 会不断累积（写满就再开一个），合并又会为数据块创建新的空白块，所需的物理块数会超过初始占用的两个数据块。如果把图中画的 4 个块当成全部物理块，更新序列会因为既没有空闲块开新 log block、也没有空闲块作为 full merge 的落点而卡死。图中那 4 个块只是初始的两个数据块加两个空闲块，用来交代初始状态，并不是说明设备只有 4 个块。正确的读法是：空闲块数量充足（可视为无限），first-fit 时总能取到物理块号最小的空闲块。

问题二：缺少 FAST 的关键参数，尤其是 victim（牺牲）算法

要把 FAST 的写放大、擦除次数、最终状态算成唯一答案，至少还需要交代：同时最多允许几个 log block；是否区分 SW 与 RW log block；何时触发合并；以及最关键的——当 log block 用满需要回收时按什么规则选 victim（原始 FAST 用 round-robin）。题面只给了“log table 最多 3 条记录”和“连续写入一个 block 不论来自哪个 block”，既没说 log block 上限，也没说 victim 规则，因此无法确定逐步过程，四个小问都不唯一。下面给出补齐了这些参数的严格版本。

FAST 的严格版本题目

SSD 的 FTL 采用 FAST 管理逻辑地址到物理地址的映射，约定如下。

每个 block 含 4 个 page， $\text{page id} = 4 \times \text{block id} + \text{页内偏移}$ 。block table 记录逻辑 block 到物理 block 的块级映射，log table 记录当前位于 log block 中的有效逻辑 page 到物理 page 的页级映射。

更新一律写入全关联（fully-associative）的 log block：更新 page 按到达顺序连续追加写入当前 log block，不论它属于哪个原始 block。本题只使用这种 random log block，不单独设置 sequential log block。

系统同时最多保留 2 个 log block。当前 log block 写满后，若活跃 log block 数未达上限，则新开一个空白 log block 继续追加；若已达上限且仍需空间，则按 round-robin（先进先出）选最早开启的 log block 作为 victim 回收，回收完再新开 log block。

回收一个 victim log block 时，对其中含有有效页的每一个原始数据块各做一次合并：把该数据块的全部最新页（可能分散在 victim、其他 log block、旧数据块中）收集到一个新的空白 block，更新 block table，擦除旧数据块；victim log block 在它涉及的所有数据块都合并完成后被擦除。

空白 block 分配采用 first-fit（取物理块号最小的空闲块）。空闲 block 数量充足，不限于图中画出的 4 个。

初始状态：block table 为 $1000 \rightarrow 0$ 、 $2000 \rightarrow 4$ ；block 0 = a, b, c, d；block 1 = e, f, g, h；其余块为空，log table 为空。页内偏移为 a/e=0、b/f=1、c/g=2、d/h=3。更新序列为

e, f, a, a, a, g, g, f, h, e

求：1) 写放大为多少个 page？2) 一共发生几次擦除操作？3) switch、partial merge、full merge 各发生几次？4) 画出序列完成后的系统状态（log table、block table 与闪存中的数据块），并标出失效页。

关于写放大的口径

“写放大为多少个 page”按课件 merge 代价里“搬移 pages”的口径计：full merge 计被收集进新块的整块页数。若改用“只统计被白搬的未更新页”这一口径，数值会不同，需在评分时统一约定。本文件配套答案采用前一种口径。